

Task 1: Project Completion and Final Implementation

Requirements refinement (not scope change)

- NFR-09's cold-start threshold was changed from a placeholder of <2000ms to <500ms once real data (~75-100ms typical cold-start) showed the original threshold was too lenient.

```
//NFR-09: cold-start (model load + first inference) latency, tracked separately
if (coldStartLatencyMs.HasValue)
{
    report.NfrResults.Add(new NfrCheckResult
    {
        RequirementId = "NFR-09",
        Description = "Cold-start latency (model load + first inference)",
        Passed = coldStartLatencyMs.Value < 500.0, //not placeholder anymore, data has been analysed
        ActualValue = $"{coldStartLatencyMs.Value:F2}ms",
        Threshold = "<500ms"
    });
}
```

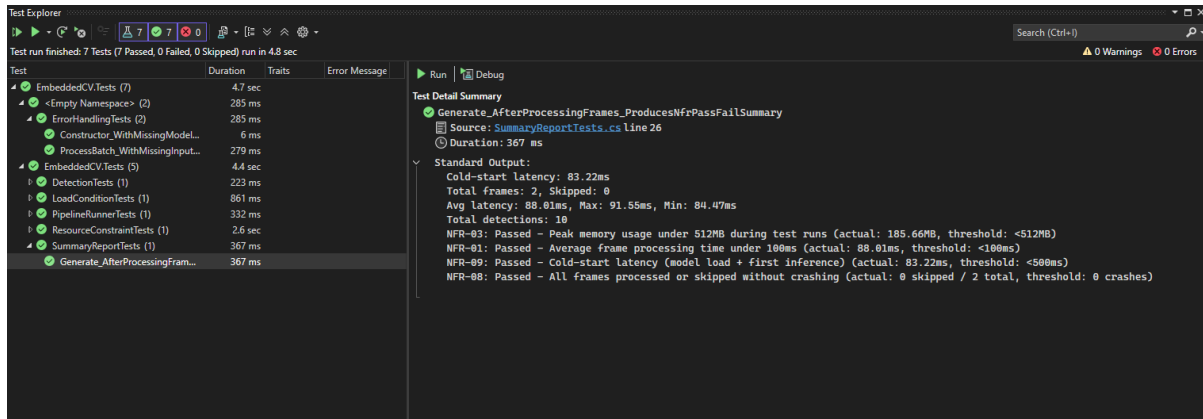
- NFR-02's threshold (≥ 15 FPS for 95% of frames) was implemented and tested against real data, revealing the system always achieves ~10-12 FPS under baseline CPU-only inference which is an NFR failure, not an implementation gap/ This brings the question of whether 15 FPS is a realistic target for CPU-bound embedded-simulated inference, which we will revisit during quality metrics and release evaluation (Task 8) rather than treat it as a defect to silently fix by lowering the threshold.

The screenshot shows the Visual Studio Test Explorer interface. On the left, a tree view lists various tests under 'Test Explorer'. The 'Generate_WithLargerFrameBatch...' test is highlighted, showing a duration of 1.9 sec. On the right, the 'Test Detail Summary' for this test is displayed. It shows the test passed, with a duration of 1.9 sec. The 'Standard Output' section lists 20 frames with their respective FPS values, ranging from approximately 11.2 FPS to 12.3 FPS. At the bottom, a summary line states: 'NFR-02: Failed - At least 95% of frames sustain minimum 15 FPS (Actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >=15 FPS)'.

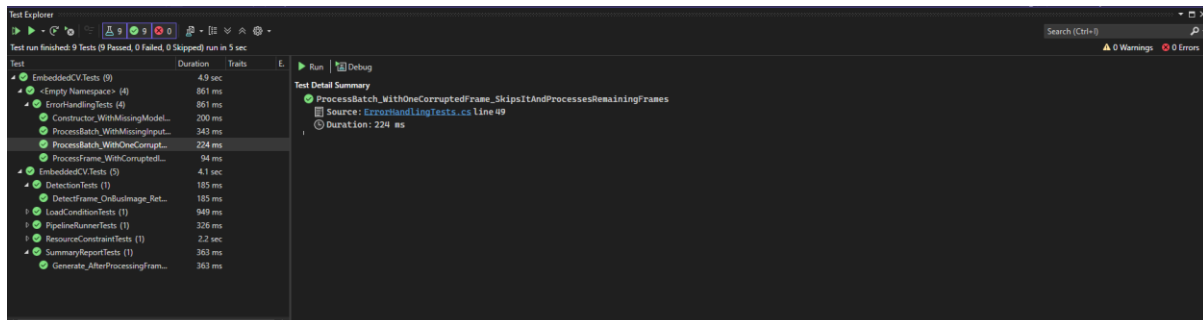
Frame	Duration (ms)	FPS
1	89.25ms	11.2 FPS
2	89.92ms	11.1 FPS
3	85.40ms	11.7 FPS
4	87.58ms	11.4 FPS
5	88.81ms	11.4 FPS
6	91.31ms	11.0 FPS
7	85.96ms	11.6 FPS
8	89.23ms	11.2 FPS
9	84.96ms	11.8 FPS
10	81.98ms	12.3 FPS
11	90.09ms	11.1 FPS
12	85.76ms	11.7 FPS
13	96.37ms	10.4 FPS
14	83.21ms	12.0 FPS
15	90.71ms	11.0 FPS
16	81.18ms	12.3 FPS
17	93.27ms	10.7 FPS
18	88.66ms	11.4 FPS
19	90.53ms	11.0 FPS
20	90.22ms	11.2 FPS

Implementation additions (filling gaps seen in Assessment 1 feedback)

- NFR-03 (memory usage) previously had a defined threshold with no implemented measurement; this has been implemented using process working set memory sampling and is now actively tested

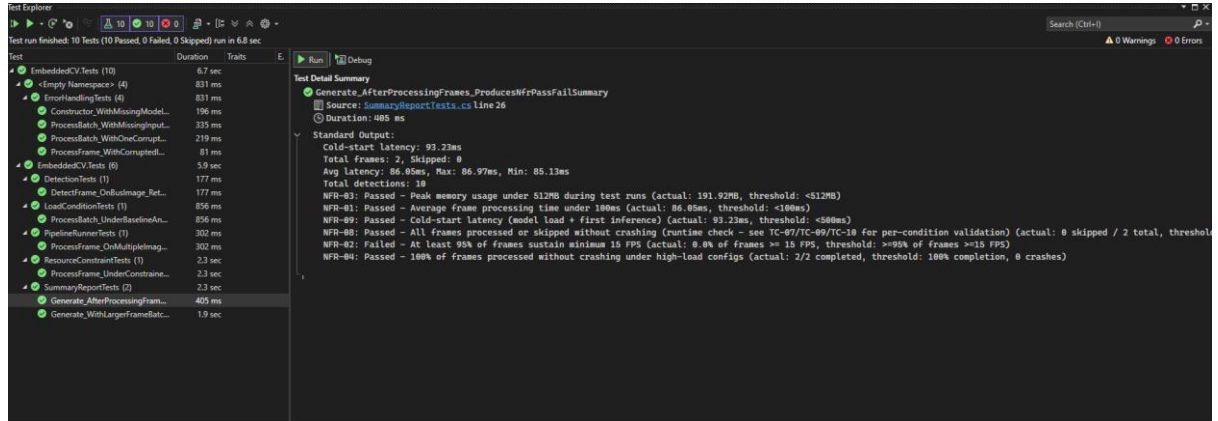


- NFR-08 (error handling) previously only verified that no unhandled exception occurred during a run; this has been changed with four dedicated test cases validating each specific error condition individually (missing model, corrupted frame, invalid input path), addressing the “false pass” risk seen in Assessment 1 feedback. Under `ErrorHandlingTests` are the four tests, covering Missing model (FR-09), Missing/invalid input path (FR-11), Single corrupted frame isolated (FR-10), and corrupted frame within a batch, confirming continuation (FR-10).

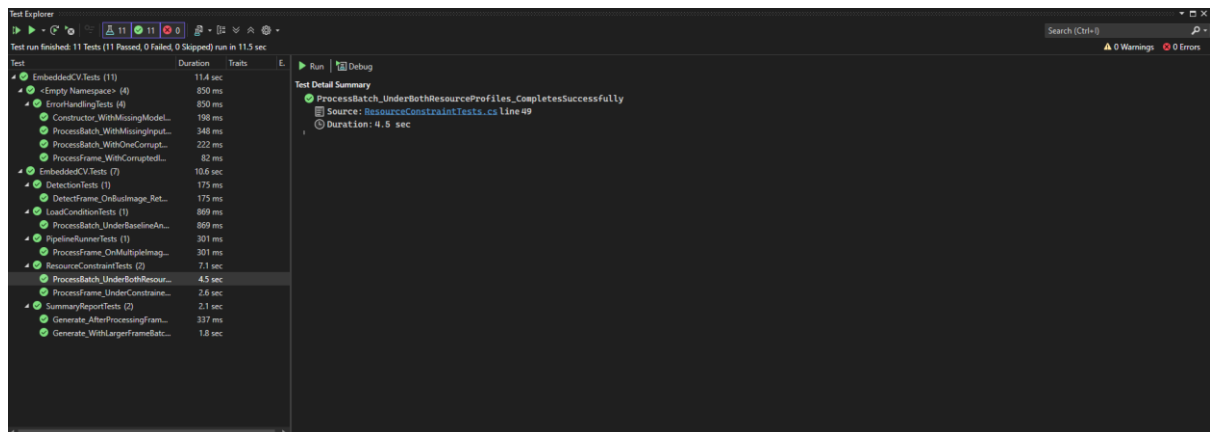


- NFR-04, NFR-06, and NFR-07 previously untested, now have dedicated automated test coverage confirming reliability under load, consistency across repeated runs, and portability across hardware profiles.

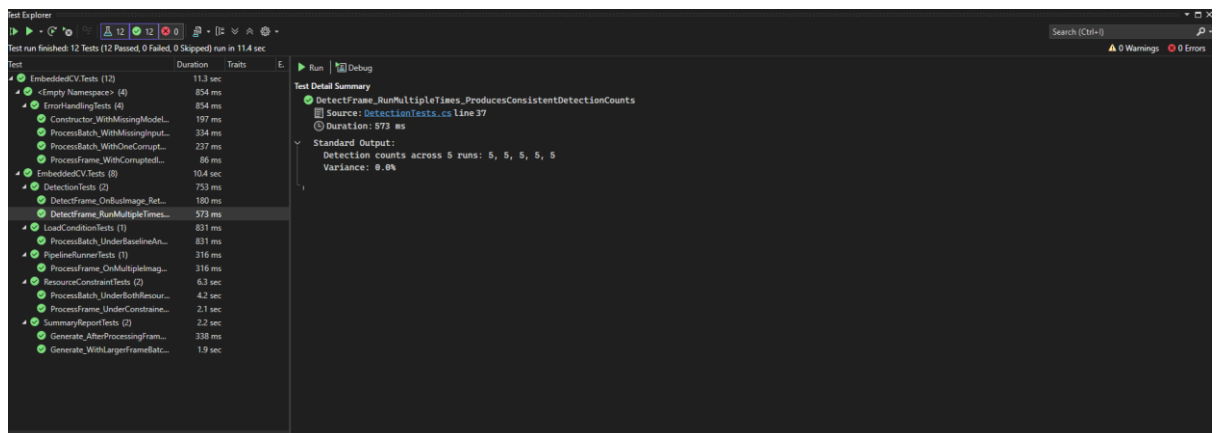
NFR-04



NFR-06



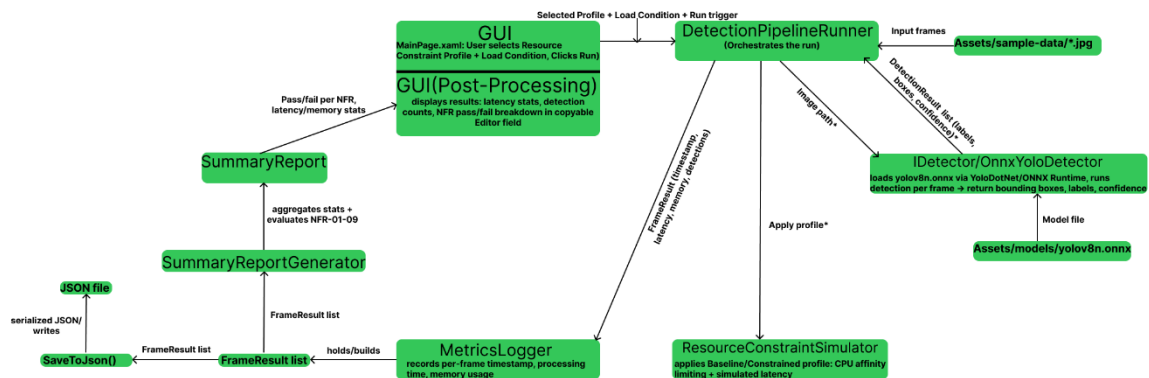
NFR-07



Architectural or technological changes

The core architecture (GUI → DetectionPipelineRunner → Detector/ResourceConstraintSimulator/MetricsLogging → SummaryReportGenerator) and the stack (C#/.NET 9, MAUI Windows, YoloDotNet/ONNX Runtime, MSTest) are unchanged from Assessment 1. Development since then is focused on finishing test coverage and measurement gaps seen in feedback, not restructuring the system.

Architecture/Data Flow diagram



*: For the setup of those marked, Image path and DetectionResult list are per-frame, repeated calls
But Apply profile is a one-time setup call (before the per-frame loop begins)

GUI(Profile: Baseline, Load condition: Baseline)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Baseline

Load Condition

Select load condition

Baseline

Run Test

Run complete.

Results

Profile: Baseline | Load: Baseline

Frames: 2 (Skipped: 0)
Avg latency: 87.84ms
Max latency: 88.16ms
Min latency: 87.52ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 300.27MB, threshold: <512MB)
NFR-01: PASS — Average frame processing time under 100ms (actual: 87.84ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >=15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

GUI(Profile: Baseline, Load condition: High load)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Baseline

Load Condition

Select load condition

High Load

Run Test

Run complete.

Results

Profile: Baseline | Load: HighLoad

Frames: 2 (Skipped: 0)
Avg latency: 136.57ms
Max latency: 164.49ms
Min latency: 108.65ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 312.88MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 136.57ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >=15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

GUI(Profile: Constrained, Load condition: Baseline)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Constrained (Embedded-like)

Load Condition

Select load condition

Baseline

Run Test

Run complete.

Results

Profile: Constrained (Embedded-like) | Load: Baseline

Frames: 2 (Skipped: 0)
Avg latency: 2673.44ms
Max latency: 2880.70ms
Min latency: 2366.18ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 318.76MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 2673.44ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >=15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

GUI(Profile: Constrained, Load condition: High load)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Constrained (Embedded-like)

Load Condition

Select load condition

High Load

Run Test

Run complete.

Results

Profile: Constrained (Embedded-like) | Load: HighLoad

Frames: 2 (Skipped: 0)
Avg latency: 3917.87ms
Max latency: 4402.50ms
Min latency: 3433.25ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 321.32MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 3917.87ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >=15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

Deferred, not implemented

A feature, considered previously, which would allow users to upload their own dataset for testing (rather than the fixed sample set) was evaluated and deliberated on by Me (Yaacoub) and Matthew, we concluded that it doesn't map to any required Assessment 2 task and was judged not a good use of our time. This is documented as a future recommendation (Task 10) rather than abandoned.

Task 2: Requirements Verification and Traceability

- Review and finalise the functional and non-functional requirements from Assessment 1.
- Update the Requirements Traceability Matrix.
- Map requirements to implemented features, test cases, test results, and defects where applicable.
- Identify any requirements that are partially implemented, unverified, or out of scope, and justify them.

Task 3: Test Execution and Final Test Results

- Execute the planned tests against the completed system.
- Include relevant functional and non-functional tests, such as unit, integration, system, acceptance, regression, usability, security, performance, or accessibility testing.
- Include positive, negative, boundary, invalid-input, and error-handling scenarios where applicable.
- Provide at least *fifteen (15)* final test cases with execution results.
- Summarise the number of planned, executed, passed, failed, blocked, and not-run tests.
- Explain failed tests, corrective actions, regression testing, and remaining quality risks.

Task 4: Test Automation and Quality Engineering

- Identify suitable test cases for automation and justify the selection.
- Implement automated tests using an appropriate tool or framework where suitable.
- Report automated test execution results using logs, screenshots, reports, or repository evidence.
- Discuss the benefits, limitations, and maintainability of the automated test suite.
- Where automation is not appropriate, justify the use of manual or exploratory testing.

Task 5: Continuous Integration and DevOps QA

- Describe the development and integration workflow.
- Implement a simple CI workflow or equivalent automated quality check where appropriate.
- Define quality gates such as successful build, passing tests, linting, code review, or coverage threshold.
- Demonstrate smoke or sanity testing where relevant.
- If CI/CD is not technically appropriate, justify an alternative quality workflow.

Task 6: Defect Management and Root Cause Analysis

- Maintain a final defect register covering defects found during development and testing.
- Define severity, priority, and defect status categories.
- Describe the defect lifecycle used in the project.
- Analyse at *least three (3)* significant defects using root cause analysis.
- Propose corrective actions and defect-prevention measures.
- Explain how defect analysis improved product or process quality.

Task 7: Quality Testing

- Conduct quality testing relevant to the project, such as security, performance, usability, accessibility, compatibility, reliability, or maintainability testing.
- Address at *least two (2)* quality areas where applicable.
- Use risk-based reasoning to justify the selected testing activities.
- Document testing evidence, results, limitations, and recommended improvements.

Task 8: Quality Metrics and Final Release Decision

- Present relevant quality metrics, such as test pass rate, defect severity distribution, requirement coverage, test coverage, automation coverage, code quality indicators, or commit activity.
- Evaluate the completed system against requirements, acceptance criteria, and quality attributes.
- Summarise testing results, defect status, metrics, limitations, and residual risks.
- State whether the system is ready for release, conditionally ready, or not ready for release.
- Justify the release decision using evidence.

Task 9: AI-Assisted Development and Critical Reflection

- Explain how GitHub Copilot was used for implementation, testing, debugging, refactoring, or QA activities.
- Provide representative evidence of AI-assisted development.
- Explain how AI-assisted outputs were reviewed, modified, tested, accepted, or rejected.
- Identify any quality risks found in AI-assisted outputs, such as incorrect logic, insecure code, poor maintainability, or unsuitable assumptions.
- Reflect on how AI-assisted development affected productivity, learning, testing, software quality, and professional practice.

Task 10: Quality Improvement and Future Recommendations

- Identify the main quality lessons learned from the project.
- Recommend improvements to the development and QA process.
- Propose future improvements to testing, automation, CI/CD, monitoring, security, usability, accessibility, or maintainability.
- Explain how the project could evolve toward stronger continuous quality engineering practice.

Task 11: Team Contribution and Professional Reflection

- Provide a contribution statement for each group member.
- Provide evidence from GitHub commits, project artefacts, testing work, documentation, and implementation tasks.
- Include a brief individual reflection from each group member.

Task 12: Project Presentation and Demonstration

Each group must prepare a 15-slide presentation and demonstrate the completed project. The presentation should clearly summarise the problem, final solution, quality assurance activities, testing evidence, and final quality evaluation. The presentation should cover:

- Project title, group members, problem, stakeholders, and scope
- Key functional and non-functional requirements
- Final prototype, implemented features, and changes after Assessment 1
- Testing approach, coverage, execution results, and key findings
- Automation, CI/CD, or equivalent quality workflow
- Defects, root cause analysis, and corrective actions

- Quality metrics, advanced testing, and release decision
- GitHub Copilot use, limitations, risks, and future improvements
- Individual contributions and lessons learned.

All students must participate in the demonstration and show the working prototype or quality improvement solution. The demo should include key features, important test cases, automated tests or CI results where applicable, defect fixes, and evidence that major requirements have been satisfied. Students must be prepared to answer questions about the project, code, testing, quality artefacts, AI-assisted development, and individual contributions.

Deliverables

- Final Project and Software Quality Assurance Report: *6000--7000 words*
- Source code and GitHub repository link; provide evidence from GitHub commits.
- Contribution statement for each group member
- Project presentation
- Turnitin Similarity Report